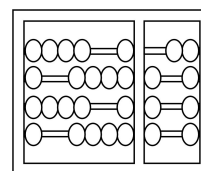




INF0093 - PROJETO
PRÁTICO COM
SISTEMAS
MULTIAGENTES - 2S
2026



PROF. MARCELO DA
SILVA REIS
msreis@unicamp.br

Entregável 1 — Especificação e Baseline

Campinas, 27 de agosto de 2026

Sumário

1	Objetivo	1
2	Formato	2
3	Especificação	2
3.1	Baseline	3
3.2	Registro da execução	3
3.3	Casos de teste e critérios	3
3.4	Análise crítica	4
3.5	Pergunta obrigatória	4
4	Entrega do notebook	4
5	Critérios de avaliação	5

1 Objetivo

Neste primeiro entregável, nosso objetivo é transformar o tema escolhido pelo grupo em uma especificação inicial de sistema e implementar um *baseline* funcional, que será usado como referência para as versões posteriores. Nesta etapa, **não é desejável construir uma arquitetura complexa**. Uma solução simples, bem especificada, executável e testável é preferível a um sistema sofisticado cuja necessidade ainda não tenha sido demonstrada¹.

¹Não há benefício em adicionar agentes, memória, planejamento ou ferramentas apenas para tornar a solução sofisticada. Cada incremento posterior deverá ser justificado por uma necessidade observada ou por uma hipótese que possa ser avaliada.

2 Formato

- Grupos de 1–3 estudantes (recomendável ao menos 2 membros por grupo).
- Entrega principal em notebook Jupyter (.ipynb), com saídas salvas.
- Documentação em células Markdown e código em células Python.
- Código executável, com instruções de configuração.

É recomendável (embora não seja obrigatório) que o grupo versione as diferentes etapas do projeto em um repositório git, como os disponibilizados no GitHub (<https://github.com>).

3 Especificação

O notebook deve conter:

- 3.1. título e descrição do problema;
- 3.2. motivação;
- 3.3. usuário/público-alvo;
- 3.4. objetivo do sistema;
- 3.5. cenários ou casos de uso;
- 3.6. escopo, não-objetivos e premissas;
- 3.7. entradas esperadas;
- 3.8. saídas esperadas;
- 3.9. requisitos funcionais (RF-01, RF-02, ...);
- 3.10. requisitos não funcionais e restrições (RNF-01, RNF-02, ...);
- 3.11. dados, ferramentas, APIs, documentos, serviços ou MCPs potencialmente necessários;
- 3.12. critérios preliminares de sucesso e avaliação, indicando **como cada um será medido**.

Escreva requisitos **verificáveis**: aplique o teste “consigo escrever hoje o código, ou o critério de julgamento, que decide se este requisito foi atendido?”. Por exemplo, “o sistema deve ser preciso” não é verificável; “em dez entradas de teste, extrair a data correta em pelo menos oito” é. O mesmo vale para os requisitos não funcionais: sempre que possível, associe um valor de referência (por exemplo, latência mediana abaixo de 10 segundos por consulta).

Delimitar o escopo é igualmente importante. Registrar o que ficou *deliberadamente* de fora protege o cronograma de cinco semanas e evita que a análise crítica se transforme em uma lista de tudo o que faltou.

3.1 Baseline

Implemente a solução funcional mais simples considerada adequada ao problema. Pode ser uma única chamada a um LLM, um pipeline linear simples ou uma combinação de método determinístico e LLM. O baseline não precisa ser multiagente e deve:

- receber uma entrada representativa;
- produzir uma saída útil;
- ser executável no notebook;
- incluir instruções de configuração;
- ser demonstrado em pelo menos três casos de teste;
- registrar os resultados obtidos.

Além disso, o grupo deve **classificar** o baseline como **completo** (já executa a tarefa principal), **parcial** (executa um subconjunto representativo do problema) ou **proxy/substituto** (fornece apenas uma referência aproximada), justificando: (i) por que a escolha é adequada; (ii) o que foi deliberadamente simplificado ou excluído; e (iii) como ela permitirá a comparação nas próximas versões.

Um alerta: um baseline artificialmente fraco torna inútil a comparação arquitetural do Entregável 4. Utilizem um prompt honesto, que vocês defenderiam. Se o baseline já resolver bem o problema, isso é um **resultado** e não um fracasso.

3.2 Registro da execução

Toda execução reportada deve vir acompanhada de: modelo e versão utilizados, temperatura, data da execução, versão do prompt, latência por caso, número de chamadas ao modelo e, quando disponível, contagem de tokens de entrada e saída. Sem esse registro, os resultados de hoje não poderão ser comparados com os das próximas arquiteturas. Vale notar que `temperature=0` reduz a variação, mas não garante saídas idênticas em serviços de inferência distribuída.

3.3 Casos de teste e critérios

Defina casos que possam ser reutilizados nas versões futuras. Quando aplicável, inclua casos normais, ambíguos, com informação ausente ou entrada incompleta.

Este conjunto deve ser **congelado**: os mesmos casos serão executados nos Entregáveis 2, 3 e 4, de modo que a comparação entre arquiteturas use sempre a mesma régua. Escreva os casos *antes* de ajustar o prompt do baseline; se o conjunto for alterado depois, o baseline precisa ser reexecutado. O mínimo exigido são três casos, mas recomenda-se dez ou mais: com apenas três casos, um único acerto vale 33 pontos percentuais, o que torna qualquer diferença entre versões indistinguível de ruído.

Considere medidas como correção, completude, taxa de sucesso, qualidade da evidência, taxa de erros, latência, número de chamadas e custo aproximado. Para cada critério, informe a forma de verificação adotada: verificação determinística (barata e objetiva, porém frágil a paráfrases), rubrica humana (necessária para respostas abertas, com as notas registradas) ou LLM como juiz (escala bem, mas deve ser calibrado contra alguns exemplos rotulados manualmente).

3.4 Análise crítica

Discuta:

- quais requisitos o baseline atende;
- quais ainda não atende;
- erros e limitações observados;
- entradas especialmente difíceis;
- capacidades adicionais que poderiam melhorar o sistema;
- quais limitações decorrem do modelo e quais decorrem da arquitetura;
- se alguma limitação observada decorre da forma de *medir*, e não do sistema;
- o que justificaria workflow, ReAct, memória, ferramentas/MCP, planejamento ou múltiplos agentes.

3.5 Pergunta obrigatória

Como o grupo pretende demonstrar, ao final do curso, que a arquitetura final apresenta vantagens em relação ao baseline?

4 Entrega do notebook

A entrega do `.ipynb` deverá ser feita até dia **7 de setembro (segunda-feira)**, utilizando o Google Classroom da disciplina: classroom.google.com/u/1/c/ODcwNjY5MjE0NjY2.

O notebook deve ser executado do início ao fim e **salvo com as saídas visíveis**: um notebook sem saídas não permite avaliar o baseline. Nenhuma chave de API deve estar escrita no notebook. Sugere-se nomear o arquivo como `E1_sobrenomes.ipynb`.

Apenas um membro do grupo deve fazer a submissão, embora todos recebam individualmente a nota e a correção. **Não serão aceitas entregas atrasadas.**

Como material de apoio, estão disponíveis no Classroom o notebook da Aula 2, com um exemplo completo de especificação, baseline instrumentado e avaliação, e um *template* de notebook para este entregável.

5 Critérios de avaliação

Serão adotados os seguintes critérios de correção:

Critério	Peso
Clareza e qualidade da especificação	25%
Adequação e executabilidade do baseline	25%
Casos de teste e critérios preliminares de sucesso	20%
Análise crítica e identificação de limitações	20%
Organização, documentação e reprodutibilidade	10%